25+ Javascript —

Q1 what is javascript? static

Ans JS is making the website interactive and Dynamic.

Q2 How the browser is ~~[illegible]~~ understand JS?

Ans Every Browser have Javascript engines in it which process the JS code.

Q3 what are data types in JS?

Ans A data types determines the type of values that can be stored in a variable.

Ex→ let a = 25; // number.

Q4 Types of data types→

Primitive data types→ Numbers, Strings, Boolean, undefined, NULL.

Non-primitive ~~Secondary~~ data types→ object, Array, function, Date, RegExp.

* Primitive data types are immutable (we cant change the value of it).

Q5 Diff. b/w Primitive and non-primitive data types.

Ans

Primitive	non-primitive
(i) num, str, Bool, undefined, null.	(i) objects, Array, fun, Date, RegExp
(ii) Primitive hold only single value.	(ii) it hold multiples value ~~[illegible]~~
(iii) Immutable	(iii) mutable
(iv) simple data types	(iv) complex data types

Q6 what arrays, fu and objects?

Arrays — it is a collection of values

Function→ it is used for perform a specific task or return a value.

objects→ it non-primitive data types which can hold multiple values.

Q7 what is scope in JS→

local scope→ ~~variable declar~~

~~local scope~~

```
function myfun(){
    // local scope
    let a = 1;
    console.log(a);
    //output 1
}
```

but ⟹

```
myfun();
console.log(a)
// outside the fun we access a var.
```

Global scope→

```
let a = 2;
function myfun(){
    console.log(a);
}
```

And ⟹

```
myfun();
console.log(a);
```

Q8 what is the diff. b/w var, let and const in JS?

Ans var→

var is function-scoped variable, we use any where in function like → defined in if condition but we also call outside the if condition.

```
function example(){
    if(true){
        var count=10;
        console.log(count);
        //output: 10
    }
    console.log(count);
    // output: 10
}
```

Date

let → blocked-scoped variable

```
function example() {
  if(true) {
    let count = 10;
    console.log(count);
    //output: 10
  }
  console.log(count);
  //output: uncaught
  //Reference Error: count is not defined
}
```

vice-versa of var.

const →

```
const z = 20;
z = 20;
// error :→ we can't assign another
```

no. for const variable or it can't change.

Q what is loop? types of loop in JS →
types
① for, while, do-while, for-of, for-in

Q9 Diff b/w for-of and for-in loop?

Ans

```
let arr = [1,2,3];
for (let val of arr) {
    console.log(val);
}
//output → 1,2,3
```

for-of

```
// for-in
const Person = {
  name = 'John',
  age : 30,
  role : 'Developer'
};
for (let key in Person){
  cnl(Person[key]);
}
//output → John 30 developer
```

for-of	for-in
It is access each value directly without index	It is access with the keys of an object

Q10 What is for each method?
ForEach can do both "for of" & "for in".

```
Ex → array = [1,2,3];
array.forEach (function(item){
    cnl(item);
});   //output → 1 2 3

object.values(Person).forEach(value => {
    cnl(value);
});   //output: John 30 Developer.
```

It is iterate object & array with fix each element.

Q11 diff b/w == and === in JS

```
var x = 5;
var y = "5".
if (x == y)   //true
//It is convert string to number and then compare with no. type coercion)
```

```
var x = 5;
var y = "5";
if (x === y) //false
// not convert str strictly, then it is false (not type coercion).
```

Q12 what are the types of fu's? what are named and anonymous functions?

types → Named fn, Anonymous fn, fn Expression, Arrow fn, IIFE, callback fn, Higher-order fn.

```
//named fn
function sum(a,b){
    return a+b;
};
cnl(sum(5,3));
//output: 8
```

```
//Anonymous fn
const add = function(a,b){
    return a*b;
};
cnl(add(5,3));
//output: 20
```

* No name fu.

Q13 what is fu expression in JS?

```
// Named function expression -
const add = function sum(a,b){
    return a+b;
};
cnl(add(5,3));
```

Q14 what is arrow fu in JS →

```
const add = (x,y) => x+y;
cnl(add(5,3));
// o/p → 8
```

Q15 what are callback fu and where we use

use fu → Iteration, Event handling, Asynchronous operation, Higher order fn, Promises.

Callback fu → app.get("/login", (req,res) => {

});

Q16 what is use of Event Handling in JS →

click event: addEventListner('click', handler)

mouseover event: addEventListener("mouseover", handler)

so, on ---

* Event handling is a process of responding to user actions in web pages

Q17 what is Higher-order fu?

① → Take one or more fu as arguments (callback fun).

② → Return a fu as a result.

① → function hof(func){ → Higher order fu. callback fu.

```
    func();
}
hof(sayHello);
function sayHello(){
    cnl("Hello");
}
//output: "Hello"
```

Q18 what are asynchronous operations in JS?

```
cnl('Before SetTimeout
setTimeout(() => {
    cnl("Inside timeout");
    }, 2000);
cnl("After timeout");
//output →
//Before timeout
//After timeout
//Inside timout
```

→ Asynchronous operation/function

* Asynchronous operation are that do not block the execution of the code.

Date

Where we use Asynchronous operation –

(i) settimeout (ii) Promises (iii) Loading Data from an API (iv) Uploading files (v) Animation and transition.

Q19 What are promises in JS?

→ Promises is a way to handle asynchronous operation.

→ A promise represents value that may not be available yet but it will be available at some points in the future.

→ Promises can be in one of three states: Pending, resolved, or rejected.

Q20 How to implement promises in JS?

```
// create a promise using promise constructor

const mypromise = new Promise ((resolve, reject)=>
{
  // set a timeout of 1 second
  settimeout (()=>{
    // Generate a random no. b/w 0 and 9
    const random num = Math.floor (Math.random()
                        * 10);

    // resolve the promises
    if (randomnum < 5) {
    resolve ('Success! Random no: ${randomnum}');
    }
    // reject the promise
    else{
    reject ('Error! Random no: ${randomnum}');
    }
  }, 1000);
```

Page
Date

```
// use the then() method to handle the resolved
promises
mypromise.then (result => {
        cnl (result);
    })
// use the .catch() method to handle the rejected
promises // After then
.catch ((error) => {
        cnl (error);
    });
```

Where to use Promise in real application →

→ (i) API calls (ii) File handling (iii) Data fetching (iv) Event handling

~~Q21 What are classes and objects in JS~~

Q21 What is purpose of "this" keyword in JS?

Ans We use this for pointing or reffering the property of the current class.

Q22 What is hoisting in JS?

Ans
```
myfunction();   // Calling before declaration
```

Ex-1
```
function myfunction(){
    cnl ("Hello");
}
```

Ex-2
```
X=10;
cnl (X);
var X;
```

→ Hoisting is a JS behaviour where fn and variables declarations are moved to the top of their respective scopes during compilation phases.